

AI PR Review — como a estrutura funciona

Quatro revisores de IA em camadas que **não se sobrepõem** + um único comentário consolidado. **Aviso, não portão:** o humano continua dono da aprovação e do merge.

✓ Advisory — nunca aprova, nunca mergeia

✓ ZDR / self-host (Mozilla)

► DRAFT — local, não conectado

branch deep-review-tooling

1 A lineup — quem pega o quê

Um teste independente de 146 PRs mostrou que revisores de IA **complementam em vez de duplicar** — achados em grande parte únicos a um só revisor. Não é redundância — são eixos diferentes. Cada camada tem o seu trabalho.

Camada	Ferramenta	Pega	Roda onde	Bloqueia?
BUGS	Cursor Bugbot	Erros de correção: race, null, permissão, refs velhas	SaaS Cursor	não
SAST	GitHub Advanced Security (CodeQL + Semgrep)	Vulnerabilidades, injeção, segredos	GitHub	não*
DESIGN	thunder-deep-review (nosso)	Arquitetura, convenção da casa, intenção/docs, readability — o que os bots de bug não veem	Nosso CI (Bedrock/ZDR)	não
DONO	Humano	Decisões de design subjetivas + a decisão final	—	SIM

* GHAS pode ser configurado pra bloquear em *critical* via branch protection — independente desta estrutura. CodeRabbit ficou **fora de escopo por enquanto** (fácil de re-adicionar depois).

2 O fluxo de ponta a ponta

Numa PR (abertura ou push), A e B rodam. B só roda **depois** de A e só reporta o que A **não** pegou. No fim, o humano vê tudo e decide.

PR aberta / novo push

RODAM EM PARALELO

Job A — bots externos (SaaS)

Cursor Bugbot · GHAS. Rodam na infra deles, postam os **próprios** comentários/checks. Não orquestramos.

Job B — thunder-deep-review (nosso CI)

1. **Debounce 60s** — pushes rápidos cancelam de graça
2. **Espera A** — poll dos checks até terminarem (ou timeout)
3. **Skip-list** — lê os achados de A pra não repetir
4. **Revisa** o diff — só o que A deixou passar
5. **Posta review inline** por linha (estilo Bugbot)



Humano revisa tudo → aprova + mergeia

Branch protection intocada. Nenhum bot aprova ou mergeia.

☐ bots externos (SaaS) ☐ nosso (CI / ZDR) ☐ humano ☐ gatilho

3 Por que B "espera" A — sem needs:

O problema

O Bugbot **não é job do nosso workflow** — é app externo. `needs:` só encadeia jobs internos. Não dá pra declarar dependência dele.

A solução

B faz **polling da Checks API** até os checks de A ficarem `completed` (qualquer resultado) ou bater o **timeout**. Se A falhar, B roda mesmo assim. Identifica os bots por `app.id` numérico (não por nome, que muda).

A orquestração é **código determinístico** (`scripts/review-orchestrator.mjs`), não tool-call do modelo. O modelo só emite achados em JSON; todo I/O do GitHub é no código:

Passo	O quê	Toca cred?
-------	-------	------------

pre	poll dos bots → monta skip-list → calcula deep-mode	não (sem AWS)
diff	<code>git diff base..head</code> → arquivo (sem <code>Bash</code> no modelo)	não
model	lê diff + skip-list → emite findings JSON (read-only)	OIDC→Bedrock
post	valida JSON → dedup vs threads abertos → resolve corrigidos → posta review inline	não

4 O review — inline, por linha (estilo Bugbot)

B **não** posta um comentário solto. Posta um **review na PR com comentários ancorados em `file:line`** — igual ao Bugbot. Cada achado fica na linha exata, com severidade e fix. `event=COMMENT` (nunca approve/request-changes → não bloqueia).

O que ele posta

- 1 **review** com N **comentários inline** (`path` + `line`)
- cada um: severidade (blocking/convention/nit) + fix + regra citada
- ancorado ao `commit_id` = head SHA revisado
- nits agrupados/capados pra não poluir

No novo push (convergência)

- re-revisa contra o head novo
- posta só os achados **novos** (dedup vs threads próprios abertos)
- achado **corrigido** → **resolve o próprio thread** (outdated)
- não re-posta o que já está aberto

Estado **por thread**, não um post editável: B gerencia **os próprios threads** (resolve quando o código corrige), igual ao Bugbot. Resolve só os achados de B; o Bugbot cuida dos dele. Dedup é **best-effort** (semântico) — pode ocasionalmente repetir; aceito, não prometido como "zero overlap".

5 Dentro do thunder-deep-review

Skill read-only que reproduz "a régua da casa". Pra recall alto, roda como **fan-out** de revisores especialistas + um merge.

~63%

recall vs revisor humano exigente (deep mode)

75% / 65%

docs-intent / correctness (onde mais importa)

0

referências a pessoas (é a régua, não a pessoa)

7 lanes amplas

- arch** arquitetura / abstração / coupling
- conv** convenções + React (CLAUDE.md)
- corr** correctness / lógica
- err** error-handling + testes
- docs** docs-intent / completeness
- sec** security / privacy / perf
- read** readability / simplificação

6 micro-especialistas (deep mode)

- μ** dead-code / guards mortos
 - μ** async-hygiene (.then, fire-and-forget)
 - μ** naming / casing
 - μ** typos / JSDoc
 - μ** simplify / restructure
 - μ** module-surface / DRY
- + dispatcha o subagent **powersync-sync-reviewer** quando o diff toca tabela synced (pega o caso "uma tabela synced, a irmã não").

6 Automático vs humano

Automático

- Achar bugs, vulns, problemas de design/convenção/docs
- Consolidar tudo num comentário
- Re-rodar + resolver no push
- Não-bloqueante (advisory)

Humano (inalterado)

- Ler os achados
- **Aprovar** a PR
- **Mergear**
- Decisões de design subjetivas

Decisão consciente: descartamos auto-merge/auto-aprovação (sem rede de segurança + superfície de ataque). O bot **adiciona sinal**; quem decide é gente.

7 Segurança & privacidade

- **Advisory only** — `event=COMMENT`, nunca approve/request-changes/merge. Branch protection intocada.
- **Sem fork** — job gateado em PR do mesmo repo; runner nunca executa código não-confiável. Sem `pull_request_target`.
- **ZDR** — Claude via Bedrock/Vertex em infra Mozilla (OIDC, sem chaves estáticas). Código não sai.
- **Read-only** — modelo só `Read/Grep/Glob`, sem `Bash`; diff pré-computado (fecha prompt-injection).

- **Endurecido** — least-privilege, actions pinadas em SHA, `timeout-minutes`, runner efêmero.

Status: rascunho revisado por 3 modelos (GLM-5.2 + MiMo + codex). Tudo local em `drafts/bot-review/` na worktree `deep-review-tooling` — **nada conectado, nada commitado**. Antes de ligar, há 8 verificações (input names da action, SHAs, `app.id` dos bots, OIDC role, model id, runner, diff fetch, CODEOWNERS) listadas no `README.md`.